

**LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 18
“MESIN ABSTRAK”**



**DISUSUN OLEH:
JANICA PRIMA GINTING
109082500064
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026**

SOAL LATIHAN

Statement Skema Mesin Abstrak

1.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    Sisi1    int
    Sisi2    int
    NilaiKartu int
    IsBalak  bool
}

type Dominoes struct {
    Kartu    [28]Domino
    JumlahTersisa int
}

func InisialisasiSet() Dominoes {
    var d Dominoes
    idx := 0
    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            d.Kartu[idx] = Domino{
                Sisi1:    i,
                Sisi2:    j,
                NilaiKartu: i + j,
                IsBalak:    i == j,
            }
            idx++
        }
    }
    d.JumlahTersisa = 28
    return d
}

func kocokKartu(d *Dominoes) {
    rand.Seed(time.Now().UnixNano())
    for i := d.JumlahTersisa - 1; i > 0; i-- {
        j := rand.Intn(i + 1)
        d.Kartu[i], d.Kartu[j] = d.Kartu[j], d.Kartu[i]
    }
}
```

```

}

func ambilKartu(d *Dominoes) Domino {
    if d.JumlahTersisa <= 0 {
        return Domino{Sisi1: -1, Sisi2: -1, NilaiKartu: -1, IsBalak: false}
    }
    kartuDiambil := d.Kartu[d.JumlahTersisa-1]
    d.JumlahTersisa--
    return kartuDiambil
}

func gambarKartu(k Domino, suit int) int {
    if k.Sisi1 == suit {
        return k.Sisi2
    } else if k.Sisi2 == suit {
        return k.Sisi1
    }
    return -1
}

func nilaiKartu(k Domino) int {
    return k.NilaiKartu
}

func galiKartu(d *Dominoes, kartuTarget Domino) {
    ketemu := false
    fmt.Printf("Mencari pasangan kartu target (%d,%d) dengan fungsi galiKartu...\n",
kartuTarget.Sisi1, kartuTarget.Sisi2)
    for d.JumlahTersisa > 0 && !ketemu {
        k := ambilKartu(d)
        fmt.Printf("  Terambil: (%d,%d)\n", k.Sisi1, k.Sisi2)
        if gambarKartu(k, kartuTarget.Sisi1) != -1 || gambarKartu(k, kartuTarget.Sisi2)
!= -1 {
            fmt.Printf("    -> Cocok ditemukan! Kartu (%d,%d) memiliki sisi yang
sama.\n", k.Sisi1, k.Sisi2)
            ketemu = true
        }
    }
    if !ketemu {
        fmt.Println("  -> Kartu habis, tidak ada yang cocok.")
    }
}

func sepasangKartu(k1 Domino, k2 Domino) bool {
    return (nilaiKartu(k1) + nilaiKartu(k2)) == 12
}

func mainkanGaplehSederhana(d *Dominoes) {
    kocokKartu(d)
    papanKiri := -1

```

```

papanKanan := -1

kartuAwal := ambilKartu(d)
papanKiri = kartuAwal.Sisi1
papanKanan = kartuAwal.Sisi2
fmt.Printf("Permainan Gapleh Dimulai. Kartu pertama di meja: (%d,%d)\n",
papanKiri, papanKanan)

for d.JumlahTersisa > 0 {
    k := ambilKartu(d)
    if k.Sisi1 == papanKiri {
        fmt.Printf(" Memasang (%d,%d) di sisi kiri (%d)\n", k.Sisi1, k.Sisi2,
papanKiri)
        papanKiri = k.Sisi2
    } else if k.Sisi2 == papanKiri {
        fmt.Printf(" Memasang (%d,%d) di sisi kiri (%d)\n", k.Sisi1, k.Sisi2,
papanKiri)
        papanKiri = k.Sisi1
    } else if k.Sisi1 == papanKanan {
        fmt.Printf(" Memasang (%d,%d) di sisi kanan (%d)\n", k.Sisi1, k.Sisi2,
papanKanan)
        papanKanan = k.Sisi2
    } else if k.Sisi2 == papanKanan {
        fmt.Printf(" Memasang (%d,%d) di sisi kanan (%d)\n", k.Sisi1, k.Sisi2,
papanKanan)
        papanKanan = k.Sisi1
    } else {
        fmt.Printf(" Kartu (%d,%d) tidak cocok dengan meja (%d,%d), dilewati.\n",
k.Sisi1, k.Sisi2, papanKiri, papanKanan)
    }
}
fmt.Printf("Permainan selesai. State akhir meja: (%d,%d)\n", papanKiri,
papanKanan)
}

func main() {
    fmt.Println("=== PENGUJIAN MESIN ABSTRAK DOMINO ===")
    tumpukan := InisialisasiSet()
    kocokKartu(&tumpukan)

    kartuTangan := Domino{Sisi1: 4, Sisi2: 2, NilaiKartu: 6, IsBalak: false}
    galiKartu(&tumpukan, kartuTangan)

    fmt.Println("\n--- Cek Pasangan Kartu Berjumlah 12 ---")
    k1 := Domino{Sisi1: 6, Sisi2: 4, NilaiKartu: 10}
    k2 := Domino{Sisi1: 1, Sisi2: 1, NilaiKartu: 2}
    fmt.Printf("Apakah (%d,%d) dan (%d,%d) berjumlah 12? %t\n", k1.Sisi1, k1.Sisi2,
k2.Sisi1, k2.Sisi2, sepasangKartu(k1, k2))

    fmt.Println("\n--- Simulasi Alur Game Gapleh Sederhana ---")

```

```
tumpukanGapleh := InisialisasiSet()
mainkanGaplehSederhana(&tumpukanGapleh)
}
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● reddemon@reddemon-PK:/media/reddemon/New Volume/WinLix/Coding/TR/Minggu 18$ go run Soal1/Soal1.go
=== PENGUJIAN MESIN ABSTRAK DOMINO ===
Mencari pasangan kartu target (4,2) dengan fungsi galiKartu...
  Terambil: (1,1)
  Terambil: (2,3)
  -> Cocok ditemukan! Kartu (2,3) memiliki sisi yang sama.

--- Cek Pasangan Kartu Berjumlah 12 ---
Apakah (6,4) dan (1,1) berjumlah 12? true

--- Simulasi Alur Game Gapleh Sederhana ---
Permainan Gapleh Dimulai. Kartu pertama di meja: (0,6)
Memasang (0,0) di sisi kiri (0)
Memasang (1,6) di sisi kanan (6)
Kartu (4,4) tidak cocok dengan meja (0,1), dilewati.
Kartu (3,5) tidak cocok dengan meja (0,1), dilewati.
Memasang (0,2) di sisi kiri (0)
Kartu (0,5) tidak cocok dengan meja (2,1), dilewati.
Kartu (3,4) tidak cocok dengan meja (2,1), dilewati.
Kartu (0,3) tidak cocok dengan meja (2,1), dilewati.
Memasang (1,2) di sisi kiri (2)
Memasang (1,3) di sisi kiri (1)
Kartu (5,6) tidak cocok dengan meja (3,1), dilewati.
Memasang (0,1) di sisi kanan (1)
Kartu (2,4) tidak cocok dengan meja (3,0), dilewati.
Kartu (2,6) tidak cocok dengan meja (3,0), dilewati.
Kartu (1,5) tidak cocok dengan meja (3,0), dilewati.
Memasang (2,3) di sisi kiri (3)
Kartu (1,4) tidak cocok dengan meja (2,0), dilewati.
Kartu (1,1) tidak cocok dengan meja (2,0), dilewati.
Kartu (3,3) tidak cocok dengan meja (2,0), dilewati.
Memasang (2,5) di sisi kiri (2)
Kartu (4,6) tidak cocok dengan meja (5,0), dilewati.
Kartu (2,2) tidak cocok dengan meja (5,0), dilewati.
Memasang (0,4) di sisi kanan (0)
Memasang (4,5) di sisi kiri (5)
Kartu (5,5) tidak cocok dengan meja (4,4), dilewati.
Kartu (6,6) tidak cocok dengan meja (4,4), dilewati.
Kartu (3,6) tidak cocok dengan meja (4,4), dilewati.
Permainan selesai. State akhir meja: (4,4)
○ reddemon@reddemon-PK:/media/reddemon/New Volume/WinLix/Coding/TR/Minggu 18$ █

● reddemon@reddemon-PK:/media/reddemon/New Volume/WinLix/Coding/TR/Minggu 18$ go run Soal1/Soal1.go
=== PENGUJIAN MESIN ABSTRAK DOMINO ===
Mencari pasangan kartu target (4,2) dengan fungsi galiKartu...
  Terambil: (0,6)
  Terambil: (5,5)
  Terambil: (2,3)
  -> Cocok ditemukan! Kartu (2,3) memiliki sisi yang sama.

--- Cek Pasangan Kartu Berjumlah 12 ---
Apakah (6,4) dan (1,1) berjumlah 12? true

--- Simulasi Alur Game Gapleh Sederhana ---
Permainan Gapleh Dimulai. Kartu pertama di meja: (6,6)
Memasang (4,6) di sisi kiri (6)
Memasang (4,5) di sisi kiri (4)
Kartu (1,1) tidak cocok dengan meja (5,6), dilewati.
Kartu (0,4) tidak cocok dengan meja (5,6), dilewati.
Kartu (2,2) tidak cocok dengan meja (5,6), dilewati.
Memasang (5,6) di sisi kiri (5)
Kartu (3,5) tidak cocok dengan meja (6,6), dilewati.
Kartu (0,5) tidak cocok dengan meja (6,6), dilewati.
Kartu (1,3) tidak cocok dengan meja (6,6), dilewati.
Kartu (0,0) tidak cocok dengan meja (6,6), dilewati.
Kartu (2,5) tidak cocok dengan meja (6,6), dilewati.
Kartu (1,4) tidak cocok dengan meja (6,6), dilewati.
Kartu (4,4) tidak cocok dengan meja (6,6), dilewati.
Kartu (1,2) tidak cocok dengan meja (6,6), dilewati.
Kartu (2,4) tidak cocok dengan meja (6,6), dilewati.
Memasang (3,6) di sisi kiri (6)
Memasang (2,3) di sisi kiri (3)
Memasang (2,6) di sisi kiri (2)
Kartu (0,2) tidak cocok dengan meja (6,6), dilewati.
Memasang (0,6) di sisi kiri (6)
Memasang (1,6) di sisi kanan (6)
Kartu (5,5) tidak cocok dengan meja (0,1), dilewati.
Kartu (3,3) tidak cocok dengan meja (0,1), dilewati.
Kartu (3,4) tidak cocok dengan meja (0,1), dilewati.
Memasang (0,3) di sisi kiri (0)
Memasang (1,5) di sisi kanan (1)
Kartu (0,1) tidak cocok dengan meja (3,5), dilewati.
Permainan selesai. State akhir meja: (3,5)
○ reddemon@reddemon-PK:/media/reddemon/New Volume/WinLix/Coding/TR/Minggu 18$ █
```

Deskripsi Program:

Program Kode ini adalah simulasi permainan kartu domino berbasis Mesin Abstrak, di mana program membuat set awal berisi 28 kartu unik, mengocoknya secara acak, dan menyediakan fungsi untuk mengambil serta mencocokkan kartu berdasarkan angka sisinya. Di bagian akhir program langsung menguji fungsinya lewat simulasi otomatis mulai dari mencari kartu yang cocok, mengecek apakah kombinasi dua kartu berjumlah 12, hingga menyimulasikan jalannya permainan di atas meja sampai tumpukan kartunya habis.

2.

Source Code:

```
package main

import (
    "fmt"
)

type MesinKarakter struct {
    pita    string
    indeks  int
    karakter rune
    akhirPita bool
}

func (m *MesinKarakter) start(input string) {
    m.pita = input
    m.indeks = 0
    m.akhirPita = false
    if len(m.pita) > 0 {
        m.karakter = rune(m.pita[m.indeks])
        if m.karakter == '.' {
            m.akhirPita = true
        }
    } else {
        m.akhirPita = true
    }
}

func (m *MesinKarakter) maju() {
    if !m.akhirPita {
        m.indeks++
        if m.indeks < len(m.pita) {
            m.karakter = rune(m.pita[m.indeks])
            if m.karakter == '.' {
                m.akhirPita = true
            }
        } else {
            m.akhirPita = true
        }
    }
}
```

```

    }
    }
}

func (m *MesinKarakter) cc() rune {
    return m.karakter
}

func (m *MesinKarakter) eop() bool {
    return m.akhirPita
}

func algoritmaAnalisisKarakter(input string) {
    var m MesinKarakter
    m.start(input)

    totalKarakter := 0
    hurufA := 0
    kataLE := 0
    var karakterSebelumnya rune = ' '

    fmt.Printf("Membaca isi pita: ")
    for !m.eop() {
        charAktif := m.cc()
        fmt.Printf("%c", charAktif)
        totalKarakter++

        if charAktif == 'A' {
            hurufA++
        }

        if karakterSebelumnya == 'L' && charAktif == 'E' {
            kataLE++
        }

        karakterSebelumnya = charAktif
        m.maju()
    }
    fmt.Println()

    frekuensiA := 0.0
    if totalKarakter > 0 {
        frekuensiA = float64(hurufA) / float64(totalKarakter)
    }

    fmt.Println("--- Hasil Analisis Algoritma ---")
    fmt.Printf(" * Banyak karakter terbaca: %d\n", totalKarakter)
    fmt.Printf(" * Banyak huruf 'A'      : %d\n", hurufA)
    fmt.Printf(" * Frekuensi huruf 'A'      : %.4f\n", frekuensiA)
    fmt.Printf(" * Banyak kata \"LE\"        : %d\n", kataLE)
}

```

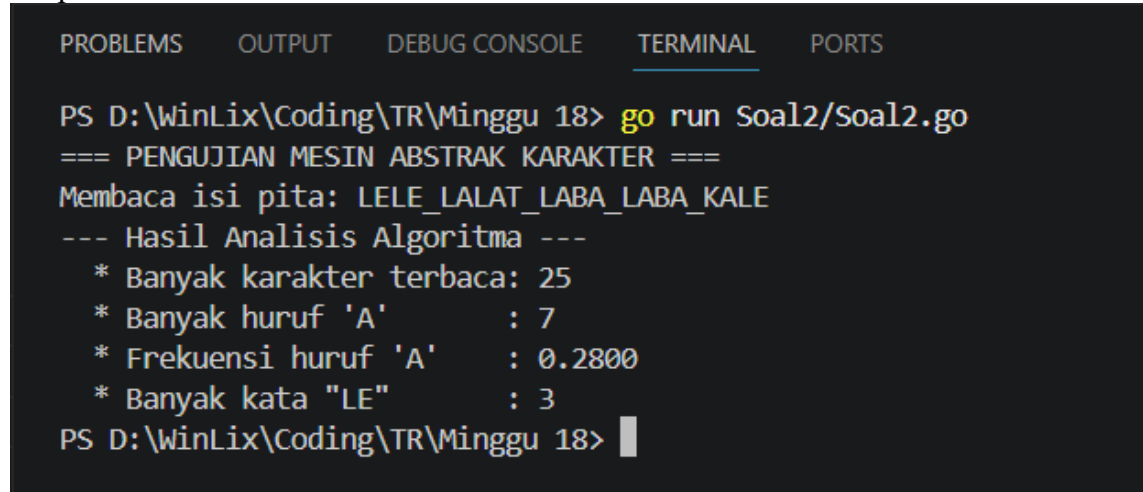
```

}

func main() {
    fmt.Println("=== PENGUJIAN MESIN ABSTRAK KARAKTER ===")
    pitaKarakter := "LELE_LALAT_LABA_LABA_KALE."
    algoritmaAnalisisKarakter(pitaKarakter)
}

```

Output:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\WinLix\Coding\TR\Minggu 18> go run Soal2/Soal2.go
=== PENGUJIAN MESIN ABSTRAK KARAKTER ===
Membaca isi pita: LELE_LALAT_LABA_LABA_KALE
--- Hasil Analisis Algoritma ---
* Banyak karakter terbaca: 25
* Banyak huruf 'A'      : 7
* Frekuensi huruf 'A'   : 0.2800
* Banyak kata "LE"      : 3
PS D:\WinLix\Coding\TR\Minggu 18>

```

Deskripsi Program:

Program Kode ini mengimplementasikan konsep Mesin Karakter Abstrak yang bertugas menelusuri untaian teks satu per satu secara berurutan. Mesin akan terus berjalan membaca huruf demi huruf dan otomatis berhenti begitu mendeteksi tanda titik “.” sebagai batas akhir pita. Sambil membaca teks, program ini juga melakukan analisis statistik pintar untuk menghitung total karakter, menghitung jumlah huruf 'A' beserta frekuensi kemunculannya, hingga mendeteksi berapa banyak kombinasi kata "LE" yang berhasil dilewati.